

Cryptography - 28_10

Exercises on Friday!

Lecture of November 4th is cancelled

By itself $\mathcal{H}_{CBC}$ is not a UFCMA MAC for VIL.

On the other hand, $\mathcal{F}(\mathcal{H}_{CBC})$ is directly secure for VIL.

We still need to cover two things for SKE:

- Combining encryption and authentication.
- Pseudorandom permutations and block ciphers design in the real world.

Start with the first, also called non-malleability.

Malleability: The ability to take some ciphertext c (of which I don't know the plain text) and change it to $\tilde{c} \neq c$, such that what's inside $\tilde{c}$ is related to the original plain text.

Concrete application: Digital Auctions.

Unfortunately, what we have seen so far is malleable. For example:

Take $c = (c_0, c_1) = (r, F_k(r) \oplus m)$ with $r \leftarrow U_n$. This is CPA secure. What kind of malleability can we do here? I can flip a bit of m .

We'll have $\tilde{c} = (c_0, c_1 \oplus 10 \dots 0)$, with the second entry being $F_k(r) \oplus (m \oplus 10 \dots 0)$

What is the definition of non-malleability? This is called chosen-ciphertext attacks (CCA) security.

✓ Definition

$\Pi = (Enc, Dec)$ is CCA secure if

$$GAME_{\Pi, \mathcal{A}}^{cca}(\lambda, 0) \approx_c GAME_{\Pi, \mathcal{A}}^{cca}(\lambda_1)$$

What is the game?

$$GAME_{\Pi, \mathcal{A}}^{cca}(\lambda, b)$$

We have an adversary $\mathcal{A}$ making queries with m . $\mathcal{C}$ answers by choosing $k \in \mathcal{K}$ and sends c . This can happen for poly times.

Then $\mathcal{A}$ sends m_0^*, m_1^* , $\mathcal{C}$ chooses b and sends the encryption c of one of the two

messages.

Then there can be another exchange of m and c .

After that, we have the adversary $\mathcal{A}$ sending $\tilde{c}$ and $\mathcal{C}$ sending back $\tilde{m}$. We have $\tilde{m} = \text{Dec}_k(\tilde{c})$, and we have $\tilde{c} \neq c$. Also this can happen for poly times.

In the end, we'll have $\mathcal{A}$ outputting b' .

Basically, we are giving the adversary decryption power, by sending a ciphertext and getting the plaintext.

We will give a general recipe for CCA-secure SKE. In particular, the following two properties will imply it:

- CPA security.
- Authenticity: The inability to create a valid ciphertext $\tilde{c}$ without knowing the secret key k .

Valid here means that $\text{Dec}(k, \tilde{c}) \neq \perp$ where $\perp$ is a special symbol.

Not every ciphertext is gonna be valid now, some of them will, others not.

✓ Definition

Π satisfies authenticity if $\forall PPT \mathcal{A}$

$$\Pr[\text{GAME}_{\Pi, \mathcal{A}}^{\text{auth}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

What is the game now?

Adversary $\mathcal{A}$ can make queries m and get c (with $c \leftarrow \text{Enc}_k(m)$), with $\mathcal{C}$ picking $k \in \mathcal{K}$, for poly times.

Then $\mathcal{A}$ sends $\tilde{c} \neq \{c\}$, and $\mathcal{C}$ outputs $1 \iff \text{Dec}_k(\tilde{c}) \neq \perp$.

✓ Theorem

$$\text{CPA} + \text{AUTH} \implies \text{CCA}$$

So if Π satisfies CPA and AUTH, then Π also satisfies CCA.

📖 Proof

We will only give a sketch of the proof.

Natural idea: Make a reduction from CPA to CCA security.

So we assume there exists A_{CCA} able to break CCA, we build A_{CPA} . The challenger is C_{CPA} .

A_{CPA} will tell A_{CCA} to start, and it will send back m , which will be passed to C . It will answer, and we'll pass it to A_{CCA} . This for poly times.

Then A_{CCA} will send $\tilde{c}$, what do we send back?

How do we "exploit" the authenticity to deal with this problem? So A_{CPA} will reply either with m or $\perp$, such that if $\tilde{c} \in \{(m, c)\}$ we return m , else we return $\perp$.
So if we have already seen c for m , we'll send the m associated with c .

Then all of this will be followed by m_0^*, m_1^* , with an answer of c^* , and then poly times m and c , and then poly times $\tilde{c}$ and $\tilde{m}$, with A_{CCA} sending b' at the end.

This suggests we should move to a hybrid $H(\lambda, b)$. In the hybrid world, we modify the way we answer to decryption queries, such that we'll never run decryption, but ask ourselves if we already know the value of m for c . This is actually identical to the CCA game, but upon input $\tilde{c}$ (a decryption query), do not run $Dec(k, \tilde{c})$. Instead: If $\tilde{c} \in \{c\}$ is the output of a previous encryption query m , return m else $\perp$.

By the above reduction to CPA, we get that $H(\lambda, 0) \approx_c H(\lambda, 1)$. *This because in H , decryption never runs, so we can have a reduction to CPA.*

It remains to prove: $G(\lambda, b) \equiv GAME_{\Pi, \mathcal{A}}^{cca} \approx_c H(\lambda, b)$.

Let BAD be the event that $\mathcal{A}$ makes a decryption query $\tilde{c} \neq c$ such that $Dec(k, \tilde{c}) \neq \perp$. *If an adversary makes a query like that, we can distinguish and we loose.*

Conditioning on $\overline{BAD}$, $G(\lambda, b) \equiv H(\lambda, b)$

$\implies$ we just need to prove $Pr[BAD] \leq negl(\lambda)$.

We'll have A_{CCA} , A_{AUTH} and C_{AUTH} .

A_{CCA} sends m , passed then to C , and then c outputted by C is passed to A_{CCA} . This for poly times. When A_{CCA} makes the query $\tilde{c}$ we are going to reply either with m or

$\perp$, this as before.

Eventually A_{CCA} will send m_0^*, m_1^* , and get c^* .

Then there can be more encryption queries and more decryption queries.

With the encryption queries, we (as A_{AUTH}), when given m , are going to give to C the value of $\tilde{c}$, but what is that value?

At the outset we'll sample $i^* \in [q]$, where q is the number of decryption queries. So $\tilde{c} = \tilde{c}_{i^*}$.

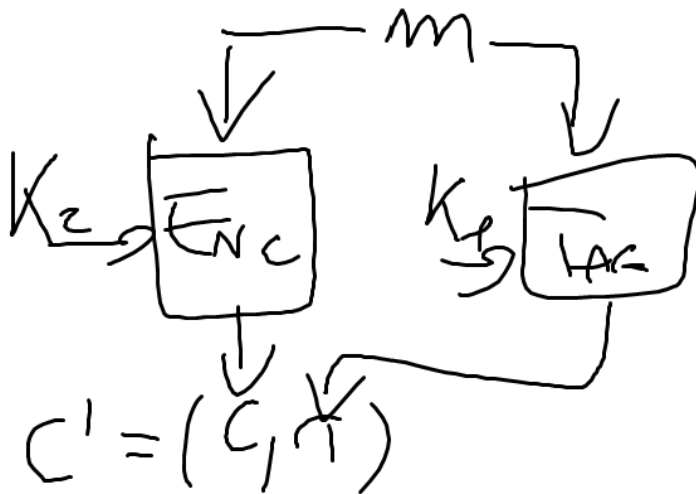
Then $Pr[A_{AUTH} \text{ wins}] \geq \frac{1}{q} \cdot Pr[A \text{ CCA provokes BAD}] \geq \frac{1}{poly}$.

■

Next natural thing is to obtain CPA+AUTH from CPA SKE and UF-CMA MACs.

There are several ways to do it:

- Encrypt-and-MAC



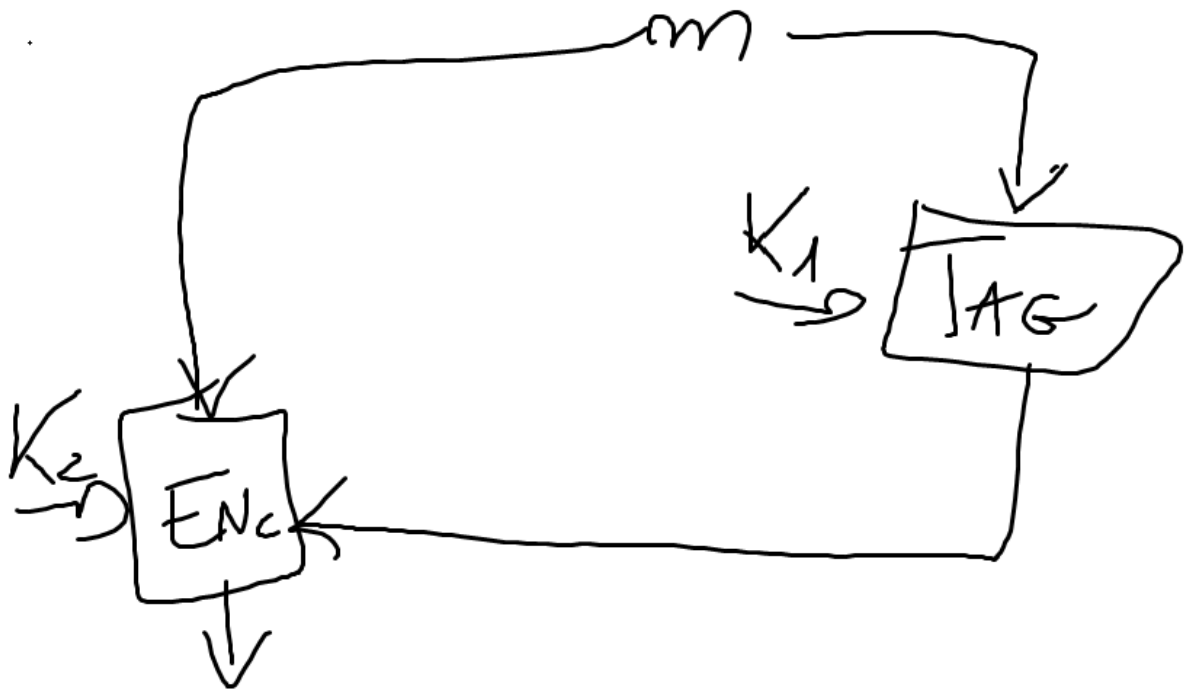
Is this a good way to do it?

🔗 Exercise

In general, it does not work.

There exist a bad $\overline{Tag}(k, m) = m[0] || \tau$ where $\tau = Tag(k, m)$.

- Mac-Then-Encrypt



❓ Exercise

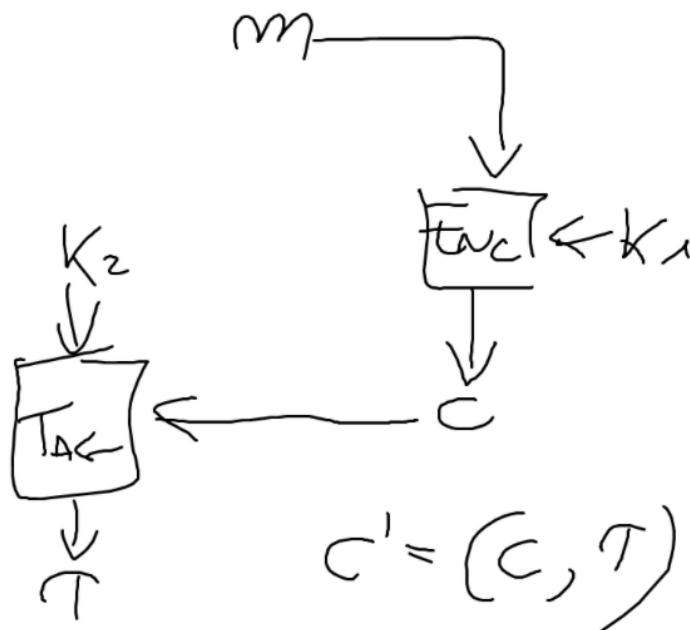
This also does not work in general. TLS does it regardless.

What we are saying is that it's not always insecure, but there are some cases in which it's not secure.

Look, $c' = \text{Enc}(K_2, m || \tau)$ is CPA secure.

There exists BAD $\overline{\text{Enc}}(k, m) = 0 || \text{Enc}(k, m)$, with Enc being CPA secure, and $\text{Dec}(k, c')$ just ignores first bit.

- Encrypt-and-MAC



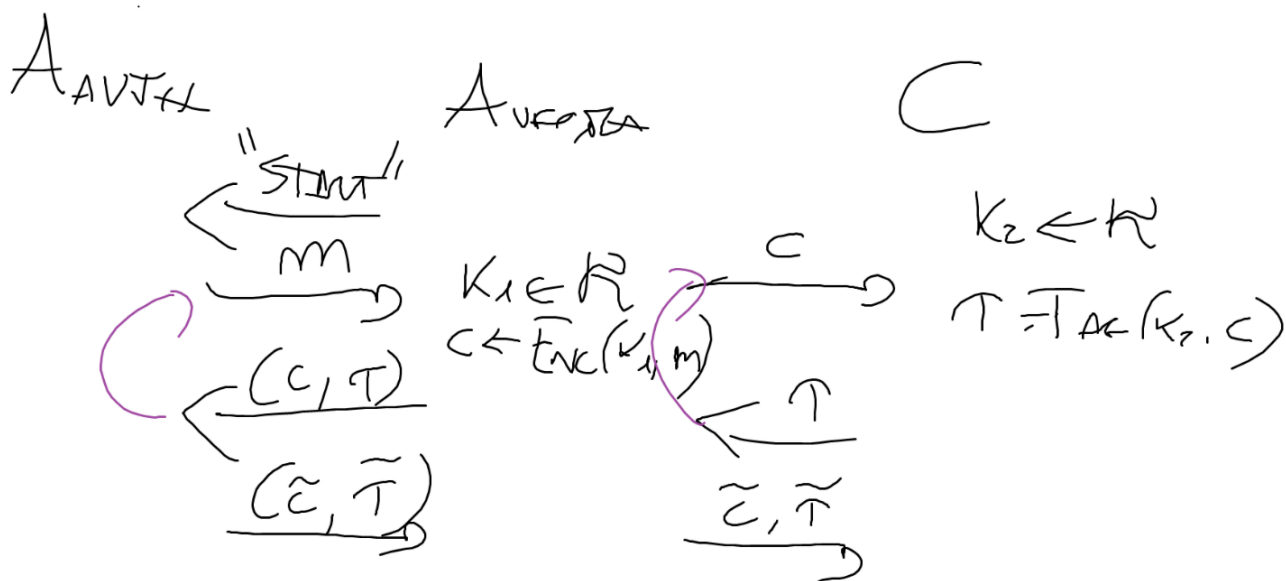
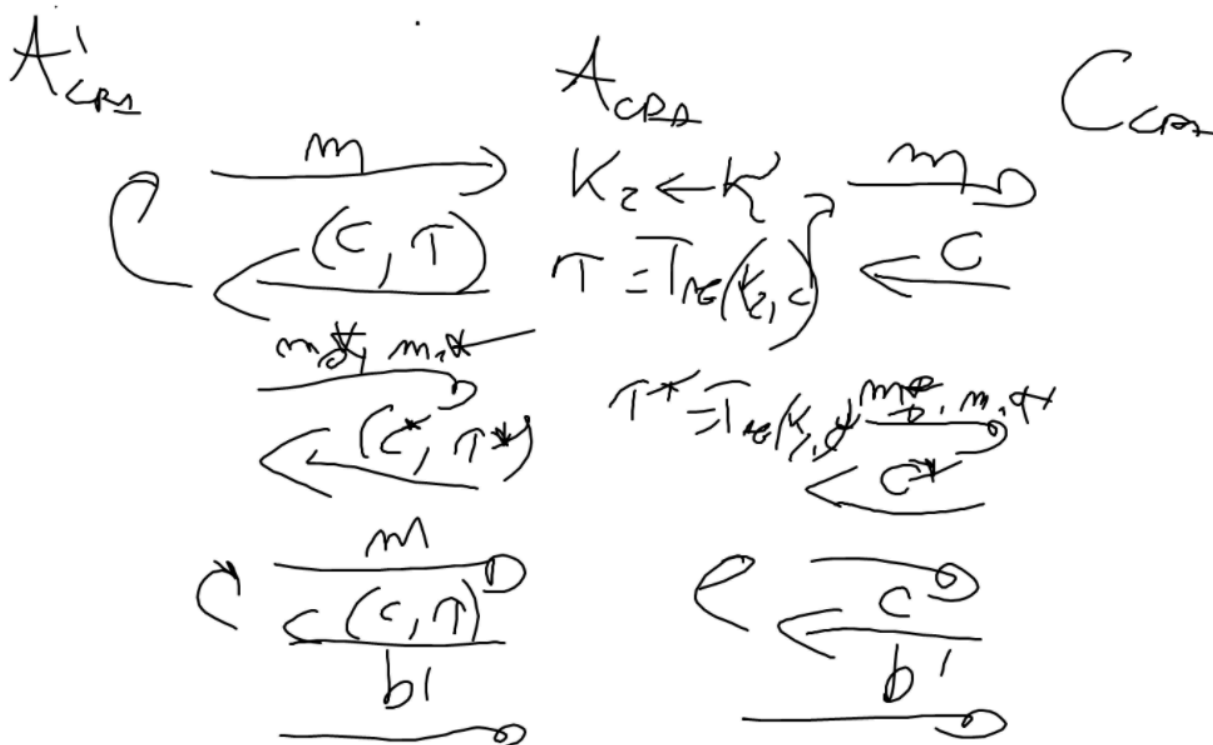
✓ Theorem

The above is CPA + AUTH assuming $\Pi_1 = (Enc, Dec)$ is CPA and $\Pi_2 = Tag$ is UFCMA with unique Tags.

① Proof

We need to show both CPA and AUTH. Basically two reductions, one to CPA security and one to UFCMA.

In the first one, we have A'_{CPA} , A_{CPA} and C_{CPA} .



The reduction wins $\iff$

- $\tilde{\tau} = \text{Tag}(k_2, \tilde{c})$
- $\tilde{c} \neq \{c\}$ but we are only guaranteed that $\tilde{c}, \tilde{\tau}$ is different from $\{(c, \tau)\}$.

It's fine as long as Tags are unique.



(Subtle remark: sometimes this is called strong UFCMA: In UFCMA $\mathcal{A}$ is allowed to forge on $m^* \in \{m\}$ but with fresh $\tau^* \notin \{\tau\}$.)